Agenda

Storyline

CFD is accurate but costly $\rightarrow$ physics-informed models reduce the cost $\rightarrow$ agentic automation removes the setup barrier $\rightarrow$ both need rigorous validation.

CFD is accurate, but expensive

Why accelerate?

- High computational cost of high-fidelity CFD.
- Expensive parameter sweeps and design iterations.
- Multi-physics coupling adds model complexity.
- Valid case set-up demands deep solver expertise.

Goal: keep the governing physics, cut cost, and make set-up accessible.

[IMAGE PLACEHOLDER]
images/cz-process-schematic.png
Silicon crystal-growth (CZ) process schematic

Application focus: two demanding CFD settings

Silicon crystal growth

- Czochralski (CZ) process.
- Coupled Navier–Stokes, particle transport and heat transfer.
- Strong thermal–fluid interaction.

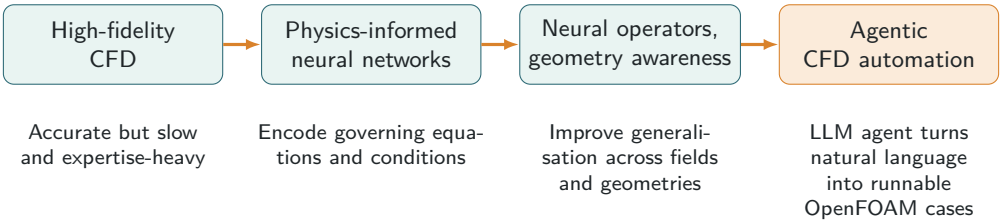
[IMAGE PLACEHOLDER]
images/cz-furnace-fields.png
CZ melt flow / temperature field

Cryogenic / hydrogen systems

- Design-oriented CFD for complex flow and thermal conditions.
- Faster surrogates support iterative design.
- Relevant to the wider space and energy use cases.

[IMAGE PLACEHOLDER]
images/cryogenic-system.png
Cryogenic tank / hydrogen system CFD

The technical story



Research question: how can physics-aware surrogates and autonomous case generation together make CFD faster and more accessible, without losing physical validity?

Physics-Informed Neural Networks (PINNs)

Core idea

A neural network approximates the solution field while the loss function penalises violations of the governing PDEs and boundary/initial conditions.

- Less dependence on large labelled datasets.
- Natural fit for multi-physics equations.
- Automatic differentiation provides PDE residuals.

Source: https://doi.org/10.1063/5.0226232

[IMAGE PLACEHOLDER]
images/pinn-concept.png
PINN concept: network, autodiff residuals, loss

PINN methodology: physics becomes part of the objective

$$L_{\text{total}} = L_{\text{PDE}} + L_{\text{BC}} + L_{\text{IC}} + \lambda L_{\text{SAL}}$$

- L_{PDE} physics-informed residual.
- L_{BC} boundary-condition loss.
- L_{IC} initial-condition loss.
- L_{SAL} sensitivity-analysis term.
- λ weighting factor.

Computation: automatic differentiation.



From PINNs to neural operators

Why move beyond a single solution?

Neural operators learn mappings between functions rather than one fixed field, improving reuse across parameter settings and related problems.

- PINO: physics-informed neural operators.
- Operator learning targets *families* of PDE solutions.
- Benefits: higher accuracy, multi-physics capability, data efficiency.



Geometry-aware operators: the next generalisation step

PI-GANO

Geometry-aware neural operators generalise across PDE parameter representations and geometries.

Source: arXiv:2408.01600v1

GAOT / MAGNO

Geometry Aware Operator Transformers and multiscale attentional graph neural operators.

Source: arXiv:2505.18781v2

[IMAGE PLACEHOLDER]
images/geometry-aware-operators.png
PI-GANO / GAOT architecture overview

Comparing the classical approaches

Approach	Primary idea	Strength	Open challenge
PINN	NN + PDE residuals	Physics constraints, data efficiency	Training stability and scaling
PINO	Operator learning + physics	Families of PDE solutions	Generalisation across settings
PI-GANO	Geometry-aware operator	Geometry generalisation	Complex geometries / representations
GAOT	Geometry-aware transformer	Attention-based operator learning	Computational cost and validation

Source: arXiv:2408.01600v1, arXiv:2505.18781v2.

Case study: CFD reference data for Czochralski growth

Reference CFD

- ANSYS Fluent 2026 R1; steady, laminar, 2D axisymmetric melt.
- Incompressible Navier–Stokes + energy, Boussinesq buoyancy, rotating walls.
- Mesh-independent grid of 8181 nodes.

Three one-parameter sweeps

Parameter	Range	Held-out case
T_{hot}	1730–1785 K	1780 K
ω_c (crucible)	-10 to -1 rpm	-4 rpm
ω_s (crystal)	4–20 rpm	7 rpm

[IMAGE PLACEHOLDER]
`images/cz-cfd-reference.png`
Reference CFD fields (T, velocity) in (r, z)

Surrogate learns $(r, z, s) \mapsto (u_r, u_z, u_\theta, p, T)$, with a whole operating condition held out.

Adaptive PINN training: what do we compare?

Three training strategies (matched settings)

- **Baseline PINN**: fixed unit weights.
- **GNPINN**: PDE and BC/IC gradients normalised separately, then summed.
- **AgenticPINN**: rule-based controller that adapts loss weights from gradient norms.

[IMAGE PLACEHOLDER]
images/controller-dynamics.png
Adaptive loss-weight dynamics (λ_{PDE}, $\lambda_{\mathrm{BC/IC}}$)

$$\mathcal{L}(\theta) = \lambda_{\mathrm{PDE}}\mathcal{L}_{\mathrm{PDE}} + \lambda_{\mathrm{BC/IC}}\mathcal{L}_{\mathrm{BC/IC}}$$

Controller drives $\lambda_{\mathrm{PDE}} \to 10^{2}$ and $\lambda_{\mathrm{BC/IC}} \to 10^{-2}$, a 10^{4} ratio.

Key point: weighted totals are not comparable across methods; report components separately.

"AgenticPINN" is a rule-based controller, not an LLM agent; unrelated to Part 4.

Result: low PDE residual $\neq$ accurate solution

Problem	Trainer	PDE loss	BC/IC loss	Rel. L_2 error
Heat equation	Baseline	3.96×10^{-3}	1.15×10^{-3}	8.43×10^{-2}
Heat equation	GNPINN	9.74×10^{-3}	$\mathbf{9.05 \times 10^{-4}}$	$\mathbf{5.41 \times 10^{-2}}$
Heat equation	AgenticPINN	$\mathbf{8.22 \times 10^{-7}}$	2.00×10^{-1}	1.368
Crystal growth	Baseline	9.26×10^{-6}	1.63×10^{-5}	–
Crystal growth	GNPINN	1.61×10^{-5}	1.01×10^{-4}	–
Crystal growth	AgenticPINN	$\mathbf{1.14 \times 10^{-6}}$	1.29×10^{-2}	–

Take-away

AgenticPINN gives the smallest PDE residual in both problems, but pushes error into the boundary/initial terms. On the heat benchmark, GNPINN is the most accurate.

Reporting recommendation

Keep unweighted PDE and BC/IC losses and an external error (where available) next to any aggregate score.

Result: CFD surrogate accuracy depends on field and sweep

GP surrogates, held-out operating condition (NRMSE)

Temperature field, by sweep	NRMSE
Hot-wall temperature	5.61%
Crucible rotation	6.34%
Crystal rotation	9.58%
Crystal-rotation sweep, by field	**NRMSE**
Swirl velocity u_θ	2.93%
Radial velocity u_r	22.95%
Axial velocity u_z	41.46%

[IMAGE PLACEHOLDER]
images/surrogate-nrmse-overview.png
Field-wise NRMSE for the three holdouts

Message

Temperature is predicted well; meridional velocities under crystal rotation are not. Aggregate scores hide this.

The OpenFOAM expertise barrier

Why CFD is hard to access

- OpenFOAM offers unmatched flexibility for physics simulation.
- Requires mastery of dozens of **C++ dictionaries**.
- Syntax is **strict and deeply interconnected**.
- Going from physics idea to working simulation is tedious and error-prone.

Where generic LLMs fall short

- Invent non-existent OpenFOAM keywords.
- Inconsistent boundary conditions across files.
- Lack physical understanding of flow regimes.

The vision

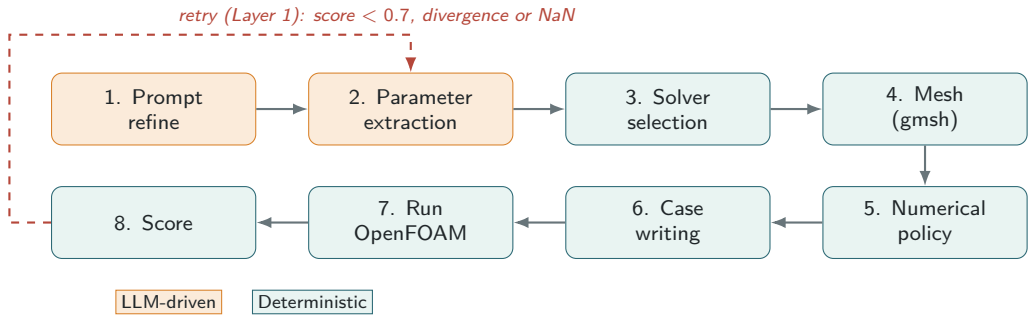
"Simulate a turbulent flow over a NACA0012 airfoil at Re= 10^6"

⇓

Full, convergent OpenFOAM case generated autonomously

Related work: FoamGPT, OpenFOAMGPT, Foam-Agent 2.0.

AutoFOAM: the eight-stage pipeline



LLM only where needed

Prompt refinement, parameter extraction, RAG context, retry generation.

Deterministic where correctness matters

Solver routing, mesh generation, case writing, execution, scoring.

Key design choices

Constrained parameter extraction

- **vLLM** for high-throughput inference.
- **xgrammar** JSON schema enforces the `CFDParams` structure.
- Output: geometry, Re, fluid, solver, turbulence model.
- Regex check of $Re = UL/\nu$ consistency.

Deterministic solver routing

- Decouples solver choice from the LLM.
- Physics flags: transient, compressible, thermal, multiphase.
- Routes to one of **7 approved solvers**.
- Avoids LLM bias, e.g. `simpleFoam` for transient vortex shedding.

Reward $r \in [0, 1]$

$+$ convergence 0.40, residual $< 10^{-4}$ 0.20, correct solver 0.10, mass conservation 0.05, valid BCs 0.05.
$-$ mesh quality, slow runtime, stagnation (up to -0.25).

Training data: static corpus + dynamic capture

Foundational knowledge corpus

- **402 rows** of expert OpenFOAM trajectories.
- 252 unique prompts across all solver types and geometries.
- Accepted trajectories: score $r \geq 0.5$.

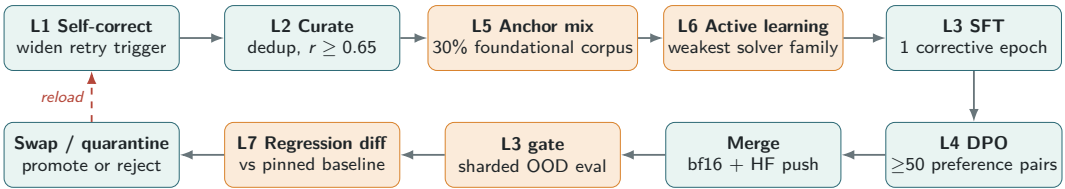
Dynamic trajectory capture

- Live production runs feed the dataset continuously.
- High-success runs ($r \geq 0.65$) join the training pool.
- Currently **210 de-duplicated topologies**.
- Failed runs logged by type for DPO mining.

Out-of-distribution benchmark: 110 prompts

Solver family	Count	Share
`simpleFoam` (steady incompressible)	74	67%
`rhoSimpleFoam` (steady compressible)	9	8%
`interFoam` (multiphase VOF)	7	6%
Others (`pimpleFoam`, ...)	20	18%

The seven-layer self-evolution loop



L5: anchor mixing	L6: active learning	L7: regression firewall
30% foundational corpus in every retraining batch prevents catastrophic forgetting.	Targets the weakest solver family and auto-generates new seeds for it.	Word-by-word diff against a pinned baseline before any weights are promoted.

Orange boxes mark safeguards; the diagram is a simplified linear view of the loop.

Results: autonomous case generation

Coverage

- **7 solver families**, e.g. `simpleFoam`, `pimpleFoam`, `rhoSimpleFoam`, `interFoam`, `icoFoam`.
- **13 gmsh geometries**: cavity, step, NACA, cylinder, periodic hill, S-bend, Ahmed body, ...
- Input: one natural-language sentence.
- Output: correct `polyMesh` boundaries and a convergent solution.

OOD headline metrics (110 prompts)

Metric	Value
FOAM FATAL-free	**110/110**
Execution success	**100%**
Solver routing accuracy	**96.4%**
Mean reward r	**0.64**
Geometry templates	**13/13**
Solver families	**7/7**

Misrouted: 4/110

Ambiguous low-Re cases only, e.g. transient cylinder at $Re = 250 \rightarrow$ `icoFoam` (physically justifiable).

Representative OOD prompts

Tag	Prompt	Predicted solver	Match
`cav_re150`	*Cavity flow Re=150 in water, 0.4 m square*	`simpleFoam`	✓
`pipe_lam`	*Laminar pipe flow Re=500, diameter 0.02 m, water*	`simpleFoam`	✓
`naca_re1m`	*NACA0012 at Re=10^6, 5° angle of attack*	`simpleFoam`	✓
`dam_tall`	*Tall dam break, 2 m wide, 5 m water column collapse*	`interFoam`	✓
`cdnoz_ma04`	*Convergent-divergent nozzle, Mach 0.4 throat, air*	`rhoSimpleFoam`	✓
`cyl_re250`	*Transient cylinder Re=250 in water, D=0.1 m*	`icoFoam`	×

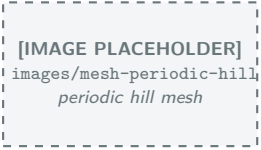
Evaluation infrastructure

8-way sharded cluster of **NVIDIA H100** GPUs; 110 OOD prompts kept isolated from training data.

Autonomously generated meshes and flow solutions

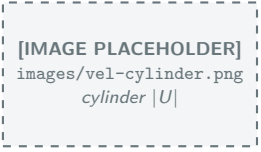


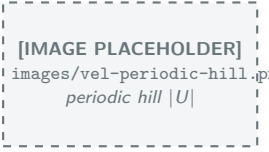














NACA 4-digit airfoil | Cylinder in cross-flow | Periodic hill | S-bend duct

Top row: meshes. Bottom row: velocity magnitude. All generated end-to-end from a single natural-language prompt, with no manual intervention.

Live demo

[IMAGE PLACEHOLDER]
`images/demo-thumbnail.png`
Video thumbnail / screenshot of the AutoFOAM demo (click opens YouTube)

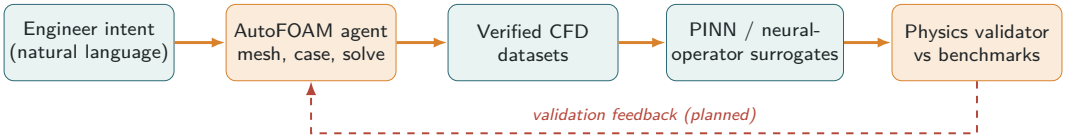
What the demo shows

- Natural-language prompt input.
- Automated parameter extraction.
- `gmsh` mesh generation.
- OpenFOAM case file synthesis.
- Live solver run and convergence.
- Velocity contour output.

Watch online

`youtu.be/Kx17bfFlMSc`

How the pieces fit together (proposed workflow)



Agent	Surrogates	Validator
Removes the set-up barrier and produces verified, reproducible cases at scale.	Physics-informed and geometry-aware models turn those cases into fast predictors.	Benchmark checks catch solutions that converge numerically but are physically wrong.

Agent and surrogate results are shown separately in this talk; their coupling is a proposed direction.

Limitations and validation gaps

Agentic CFD (AutoFOAM)

- **Geometry**: 13 parameterised gmsh models; no raw CAD (STEP/IGES).
- **Physics**: RANS only; no LES/DNS.
- **Reward**: biased toward convergence; a converged case can still be physically wrong (e.g. wrong drag coefficient).
- No cross-check against benchmark data (e.g. Ghia cavity).

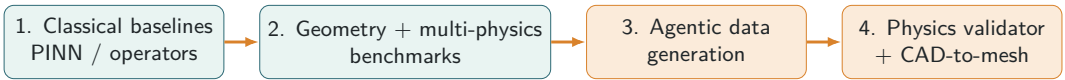
Physics-informed surrogates

- Comparisons use one matched initialisation and two physics settings.
- Simplified crystal-growth PINN has no analytical reference.
- CFD cases are steady, laminar, axisymmetric, not industrial-scale melt flow.
- Velocity components can be much harder than temperature.

Common remedy

Add an external physics validator, and report component-level and field-wise errors rather than a single aggregate score.

Research and validation roadmap



Benchmark dimensions

- Accuracy, conservation, constraint satisfaction.
- Training / inference time.
- Data and memory needs.

Validation settings

- Silicon crystal growth.
- Turbulent-flow examples.
- Cryogenic / CFRP vessel examples.

Agentic extensions

- LES/DNS numerical policy.
- More solvers (`reactingFoam`, `chtMultiRegionFoam`).
- Tighter DPO feedback from the validator.

Key takeaways

1. **CFD acceleration is the driver:** high-fidelity multi-physics simulation is expensive and expertise-heavy.
2. **PINNs add physics constraints:** PDEs, boundary and initial conditions become part of learning, but must be evaluated component by component.
3. **Operators add generalisation:** PINO, PI-GANO and GAOT target solution families and geometries.
4. **Agentic workflows remove the set-up barrier:** AutoFOAM reaches 100% execution success and 96.4% solver-routing accuracy on 110 OOD prompts.
5. **Validation closes the loop:** physics validators and benchmark checks are essential for both surrogates and agents.

Physics + geometry + agentic automation
$\longrightarrow$ a coherent path toward accelerated CFD

Selected references

- *PINN review*, https://doi.org/10.1063/5.0226232.
- *PI-GANO: physics-informed geometry-aware neural operator*, arXiv:2408.01600v1.
- *Geometry Aware Operator Transformer (GAOT) / MAGNO*, arXiv:2505.18781v2.
- *PINNsFormer: A Transformer-Based Framework for Physics-Informed Neural Networks* (2023), arXiv:2307.11833v3.
- D. Kolberg, C. Anders, S. Reke (2023), silicon VMCz crystal-growth PINNs, doi:10.1080/00219592.2023.2236656.
- J. Ling, Y. Zhang, D. Chen (2023), thermal–fluid coupling in silicon crystal growth, doi:10.1063/5.0123811.
- Y. Zhao, H. Liu, J. Xu (2025), thermal–fluid coupling in Czochralski silicon growth, doi:10.1063/5.0271778.
- N. Chapagain, R. Raj, B. K. Shine, A. A. S. (2026), *Component-Level Evaluation of Adaptive PINN Training for CFD-Oriented Crystal Growth Simulation*, NeurIPS 2026 Sim2Science workshop.
- A. G. Neelan, A. Seshaditya, *AutoFOAM: The Self-Refining Autonomous OpenFOAM Agent*.
- Related LLM-for-CFD work: FoamGPT, OpenFOAMGPT, Foam-Agent 2.0.

Resources: `github.com/AGN000/AutoFOAM`, `github.com/AGN000/FoamAgentCases`, `hf.co/arungovindneelan/foam-cfd-unified-14b`